התחיל ב:	1/01/2023, 16:48
מצב	הסתיים
הושלם ב-	1/01/2023, 16:48
הזמן שלקח	8 שניות

שאלה **1**

לא נענה

ניקוד השאלה: 4.00

בחרו את סיבוכיות זמן הריצה של הפונקציה **`foo(n)`** כפונקציה של **`n`**.

```
def foo(n):
    if n < 1:
        return 2**n

    def b(x):
        my_list = []
        for i in range(n):
            for j in range(n):
                my_list.append(i**2)

        return 17
    return b(n // 3) + foo(n - 2)
```

שאלה **2**

לא נענה

ניקוד השאלה: 4.00

בחרו את סיבוכיות זמן הריצה של הפונקציה **`booya(n)`** כפונקציה של **`n`**.

```
def booya(n):
    if n == 0:
        return []

    ans = booya(n - 1)
    n_to_str = str(n)
```

```
    n_to_str = str(n)

    for dig in n_to_str:
        for k in range(n):
            ans += [dig]

    return ans
```

שאלה **3**

לא נענה

ניקוד השאלה: 4.00

רשמו את **כל** הפלטים האפשריים של הפונקציה **f** (יש להניח כי הקלט הוא של טיפוסים סטנדרטים בלבד):

```
def w(a, n):
    try:
        print("W", end='')
        return a + n
    except:
        print("M", end='')
        return a

def f(m, n):
    try:
        a = m * n
        print("A", end='')
        b = w(a, n)
        print("B", end='')
    except:
        c = w(m, n)
        print("C", end='')
```

```
    else:
        d = w(w(b, a), a)
        print("D", end='')
    finally:
        print("F", end='')
```

רשמו את כל הפלטים האפשרים, מופרדים על ידי פסיק (",") ללא תוספות מיותרות.
רק תשובות בפורמט הנ"ל יבדקו.

שאלה **4**

לא נענה

ניקוד השאלה: 4.00

ספרו את מספר הרשימות שנוצרות במהלך ריצת הקוד הבא. רשימות שונות בזכרון אך עם תכולה זהה יחשבו לצורך הספירה כרשימות נפרדות (שימו לב: אם רשימה מסויימת מופיעה פעמים כ-alias אין לספור אותה שנית).

כמה רשימות שונות מכיל המשתנה k באופן ישיר או עקיף (כולל הרשימה k)?

```
k = [[i]*i for i in range(1, 3)]
for i in range(1, 3):
    k = list((k*i, k+k))
```

יש להזין תשובה מספרית בלבד.

תשובה:

שאלה **5**

לא נענה

ניקוד השאלה: 4.00

הפונקציה `monotonic_descending(f, s)` מקבלת פונקציה `f` וקבוצה `s`, ומחזירה האם הפונקציה `f` מונוטונית יורדת ממש, על פני האברים של הקבוצה `s`.

סיבוכיות המקום של הפונקציה צריכה להיות קבועה, בהנחה ש-`f` לוקחת מקום קבוע.

השלימו את הקוד במקומות המסומנים (תיבת מילוי בהתאם לאינדקס המסומן בקוד):

```
def monotonic_descending(f, s):
    return all((v > w) == (______(1)______)
               for  ______(2)______
               for  ______(3)______)
```

(1)

(2)

(3)

שאלה **6**

לא נענה

ניקוד השאלה: 4.00

הפונקציה `monotonic_ascending(f, s)` מקבלת פונקציה `f` וקבוצה `s`, ומחזירה האם הפונקציה `f` מונוטונית עולה (לאו דוקא ממש), על פני האברים של הקבוצה `s`.

סיבוכיות המקום של הפונקציה צריכה להיות ביחס לגודל הקבוצה, בהנחה ש-`f` לוקחת זמן ומקום קבועים.
הדרישה לסיבוכיות זמן הריצה של הפתרון היא: `O(nlog(n))`.

השלימו את הקוד במקומות המסומנים (תיבת מילוי בהתאם לאינדקס המסומן בקוד):

```
def monotonic_ascending(f, s):
    return sorted(s)==______(1)______
```

(1)

שאלה **7**
לא נענה
ניקוד השאלה: 4.00

להלן הפונקציה הרקורסיבית `f(x, m)` ולאחריה מספר קביעות (**assertions**):

```
def f(x, m):
    for i in ______(1)______:
        if ______(2)______:
            return m + x[i]
        else:
            return f(______(3)______)

assert f([11, 12, 1, 2, 9], 0) == 35
assert f([5, 1, 12, 13, 1], 0) == 32
```

השלימו את הקוד במקומות המסומנים (תיבת מילוי בהתאם לאינדקס המסומן בקוד) כך שהקביעות (**assertions**) יתקיימו:

[שימו לב: "קביעות יתקיימו" אומר שלא תיזרק שגיאה.]

(1)

(2)

(3)

שאלה **8**

לא נענה

ניקוד השאלה: 4.00

נתונה המחלקה **BinT** המייצגת קודקוד בעץ חיפוש בינארי (**BST=Binary Search Tree**):

```
class BinT:
    def __init__(self, data, left=None, right=None):
        self.data = data
        self.left = left
        self.right = right
```

להלן מימוש חלקי לפונקציה **buildBST(arr)**. פונקציה זו מקבלת מערך **arr** שהינו רשימה ממוינת של מספרים, השלימו את המימוש כך שהפונקציה תחזיר אוביקט **BinT** שהינו שורש של עץ חיפוש בינארי בעל גובה מינימאלי.

תזכורת: **BST** הינו עץ אשר מקיים שלכל קודקוד יש עד שני בנים, וכן ערך כל קודקוד בתת-עץ השמאלי קטן או שווה לשורש, אשר קטן או שווה לערך כל קודקוד בתת-עץ הימני.

```
def buildBST(arr):
    if not arr:
        return None

    mid = len(arr)//2
    root = ______(1)______(arr[______(2)______])
    root.left = ______(3)______(arr[______(4)______])
    root.right = ______(3)______(arr[______(5)______])
    return root
```

השלימו את הקוד במקומות המסומנים (תיבת מילוי בהתאם לאינדקס המסומן בקוד):

(1)

(2)

(3)

(4)

(5)

שאלה **9**

לא נענה

ניקוד השאלה: 4.00

הגנרטור `merge_generator(a, b)` הוא גנרטור **לא רקורסיבי** המקבל שתי רשימות ממוינות וממזג את איברי שתי הרשימות ומחזיר את האיברים בסדר ממוין.

לדוגמה:

```
>>> a = [1, 3, 5]
>>> b = [2, 3, 4]
>>> print(list(merge_generator(a, b)))
[1, 2, 3, 3, 4, 5]
```

```
def merge_generator(a: list, b: list):
    while ______(1)______:
        if ______(2)______:
            yield a.pop(0)
        else:
            ______(3)______
```

השלימו את הקוד במקומות המסומנים (תיבת מילוי בהתאם לאינדקס המסומן בקוד):

(1)

(2)

(3)

שאלה **10**

לא נענה

ניקוד השאלה: 4.00

יש להשלים את הפונקציה `b` ולקשט (להוסיף `decorator`) את הפונקציה `f` כך שהפונקציה תחזיר את המחרוזת `text` ללא שינוי במקרה שבו היא לא מכילה את התבנית `pattern`.

דוגמאות:

```
>>> f('abcdeabcde','cd')
'eabcde'

>>> f('abcdeabcde','dc')
'abcdeabcde'
```

```
def a(f):
    def g(t, p):
```

```
        return f(p+t, p)
    return g

def b(f):
    def g(t, p):
        if p in ______(1)______:
            return f(______(2)______, p)
        else:
            return f(t, p)
    return g

______(3)______
______(4)______
def f(text: str, pattern: str):
    return text[text.find(pattern) + len(pattern):]
```

השלימו את הקוד במקומות המסומנים (תיבת מילוי בהתאם לאינדקס המסומן בקוד):

(1)

(2)

(3)

(4)

שאלה **11**

לא נענה

ניקוד השאלה: 15.00

מותר (וחייבים) לייצר רשימה חדשה בשביל התוצאה.

סכום רשימות

כתבו פונקציה **`sum_lists(list_1, list_2)`** אשר מקבלת שתי רשימות, **`list_1`** ו-**`list_2`**, כך שכל רשימה מייצגת מספר אי שלילי כלשהו (משמאל לימין). הפונקציה תחזיר רשימה חדשה אשר הערכים בה הם תוצאת הפעלת החיבור בין שני המספרים, המיוצגים על-ידי שתי הרשימות אשר התקבלו כקלט.

הערות:
- אין לתרגם ישירות את הרשימות למספרים.
- אין להשתמש במבני נתונים אחרים מעבר לרשימות הנתונות (ולרשימה המוחזרת אותה אתם מייצרים).
- אין לשנות את הרשימות הנתונות.
- ניתן להניח שהערך הראשון בכל רשימה לא יכלול את הספרה 0 (למעט רשימות שמייצגות את 0 עצמו, קרי: **`[0]`**).
- ברשימה המוחזרת יש להימנע מהחזרת "0"ים רצופים בתחילת הרשימה.

דוגמאות:

```
>>> list_1 = [1, 2, 3] # represents 123
>>> list_2 = [4, 6] # represents 46
>>> sum_lists (list_1, list_2)
[1, 6, 9] # represents 169

>>> list_3 = [1, 7] # represents 17
>>> list_4 = [0] # represents 0
>>> sum_lists (list_3, list_4)
[1, 7] # represents 17

>>> sum_lists(list_3, list_4) is list_3
False
```

Answer:

(penalty regime: 0 %)

Reset answer

שאלה **12**

לא נענה

ניקוד השאלה: 15.00

הקוד בראש התשובה תוקנה להתאים למה שמופיע בגוף השאלה. (תוספת פרמטר data ל __init__).

ניתן לשנות את הקוד ידנית או ללחוץ על הכפתור reset answer כדי לאתחל את התשובה למצב המקורי.

סוגריים מאוזנים

נתונה המחלקה **Node**:

```
class Node:
    def __init__(self, data, prev=None, next=None):
        self.data = data # will be either '(' or ')'
        self.prev = prev
        self.next = next
```

כתבו פונקציה **balanced(head)** אשר מקבלת מצביע, **head**, לראש של רשימה מקושרת דו-כיוונית, ובודקת האם הרשימה מייצגת סדרה של סוגריים מאוזנים. במידה וכן, הפונקציה תחזיר **True**, אחרת הפונקציה תחזיר **False**.
רשימה מקושרת דו-כיוונית הינה מאוזנת אם:
1. לכל סוגר שמאלי מתאים סוגר ימני מאותו סוג אשר נמצא בתוך אותה מחרוזת, במרחק מה מצד ימין שלו.
2. תת המחרוזת שבין שני הסוגריים הללו היא גם מאוזנת.

הערה: על הפתרון לרוץ בסיבוכיות מקום קבועה ואין לספור קדקדים.

דוגמאות לסדרות מאוזנות:

```
( )
( ( ) )
( ( ) ( ) )
( ( ( ) ) )
```

דוגמאות לסדרות שאינם מאוזנות:

```
)
( (
) (
( ) ) )
( ( ) ) ) )
( ( ( ( ) )
```

Answer:

(penalty regime: 10, 20, ... %)

Reset answer

```
ev=None, next=None):
 be either '(' or ')'


S LINE! #########
```

שאלה **13**

לא נענה

ניקוד השאלה: 15.00

גודל הכתם

כתבו את הפונקציה **`find_stain_size(mat, place)`** אשר מקבלת:

- **`mat`**, מטריצה ריבועית (טבלה ריבועית) מגודל **`N x N`**, עבור **`N>=1`**, המיוצגת על ידי רשימות, כאשר איברי המטריצה הינם 0 או 1.
- **`place`**, רשימה בגודל 2 המייצגת מיקום **`[x, y]`** של איבר בודד במטריצה. הפינה השמאלית העליונה מיוצגת על ידי המיקום **`[0 ,0]`**.

הפונקציה תחזיר מספר שלם, אי-שלילי, אשר מייצג את גודל ה"כתם" שבו נמצאת הקוארדינטה **`place`**.
כאשר "כתם" מוגדר להיות רצף של '1'-ים סמוכים במערך (אופקית, אנכית או באלכסון) כך שיש מסלול כלשהו של '1'ים, בין כל שני '1'ים בכתם.

הערה: אם **`place`** מצביע על מיקום של איבר בעל ערך '0' אז הפונקציה תחזיר 0.

למשל עבור המטריצה **`mat1`** בגודל 5x5 הבאה (לצורך הקריאות, הוספנו את האינדקסים בכל שורה ובכל עמודה):

	0	1	2	3	4
0	1	1	0	0	0
1	0	1	0	1	0
2	1	0	0	0	0
3	0	0	1	1	1
4	0	1	1	1	1

```
>>> mat1 = [[1, 1, 0, 0, 0], [0, 1, 0, 1, 0], [1, 0, 0, 0, 0], [0, 0, 1, 1, 1], [0, 1, 1, 1, 1]]

>>> find_stain_size(mat1, [0,0])
4
>>> find_stain_size(mat1, [0,1])
4
>>> find_stain_size(mat1, [1,0])
0
>>> find_stain_size(mat1, [1,3])
1
>>> find_stain_size(mat1, [0,4])
0
>>> find_stain_size(mat1, [4,3])
7
```

Answer:

(penalty regime: 0 %)

Reset answer

שאלה **14**

לא נענה

ניקוד השאלה: 15.00

חלוקת מספרים לזוגות

כתבו פונקציה **`pairs_sum(d)`** אשר מקבלת כארגומנט יחיד את המילון **d**, אשר מכיל זוגות של מפתח-וערך, כאשר כל מפתח מייצג מספר, והערך מייצג את כמות ההופעות של המספר.
הפונקציה תבדוק אם ניתן לחלק את סך כל המספרים לזוגות, כך שסכום כל הזוגות הינו שווה. במידה וניתן למצוא חלוקה כזאת, הפונקציה תחזיר **`True`**, אחרת - יוחזר **`False`**.

הערה: מותר להשתמש בפונקציות עזר מובנות של פייתון.

דוגמאות:

```
>>> pairs_sum({1:1, 7:1}) # represents 1, 7
True # a single pair (1+7)

>>> pairs_sum({0:1, 1:1, 7:1, 8:1}) # represents 0, 1, 7, 8
True # we can divide to 2 pairs: (0+8) == (1+7)

>>> pairs_sum({-3:1, 1:1, 0:2) # represents -3, 0, 0, 1
False

>>> pairs_sum({-3:2, 0:2) # represents -3, -3, 0, 0
True  # we can divide to 2 pairs: (-3+0) == (-3+0)

>>> pairs_sum({0:1, 1:2, 2:1, 6:1, 7:2, 8:1}  # represents 0, 1, 1, 2, 6, 7, 7, 8
True # we can divide to 5 pairs: (0+8) == (1+7) == (2+6)

>>> pairs_sum({0:1, 1:2, 3:1, 7:2, 8:1}) # represents 0, 1, 1, 3, 7, 7, 8
False
```

Answer:

(penalty regime: 0 %)

Reset answer

שאלה **15**

לא נענה

ניקוד השאלה: 15.00

עץ חיפוש בינארי

נתונה המחלקה **`BinT`** המייצגת קודקוד בעץ חיפוש בינארי:

```
class BinT:
    def __init__(self, data, left=None, right=None):
        self.data = data
        self.left = left
        self.right = right
```

כתבו פונקציה רקורסיבית, **trim(root, low, high)** אשר מקבלת **root** (אוביקט מסוג **BinT**), אשר מהווה שורש של עץ חיפוש בינארי (**BST** = **Binary Search Tree**), וכן **low** ו-**high**, אשר הינם מספרים כלשהם (לאו דווקא מופיעים בעץ הנתון) כך ש- **low < high**.

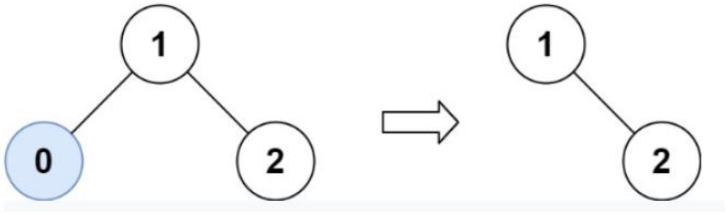
הפונקציה תחזיר **BST** "מקוצץ", אשר מכיל את כל הקודקודים המקוריים ב-**BST** הנתון (אשר כאמור, מושרש ב-**root**) וכל ערכיו הם גדולים ממש מ-**low** וקטנים ממש מ- **high**. במידה ואין קודקודים שנמצאים בטווח זה, יש להחזיר **None**.

תזכורת: **BST** הינו עץ אשר מקיים שלכל קודקוד יש עד שני בנים, וכן ערך כל קודקוד בתת-עץ השמאלי קטן או שווה לשורש, אשר קטן או שווה לערך כל קודקוד בתת-עץ הימני.

דוגמא מס' 1:

```
>>> root_1 = BinT(1, BinT(0), BinT(2))
>>> trim (root_1, low=0, high=8) == BintT(1, None, BinT(2))
True
```

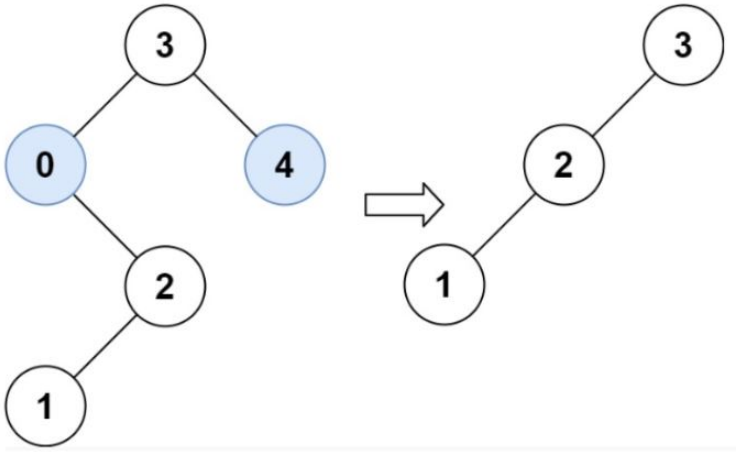
תמונה להמחשת הדוגמא:



דוגמא מס' 2:

```
>>> root_2 = BinT(3, BinT(0 ,None, BinT(2, BinT(1), None)), BinT(4))
>>> trim (root_2, low=0, high=4) == BintT(3, BinT(2, BinT(1), None), None)
True
```

תמונה להמחשת הדוגמא:



Answer:

(penalty regime: 0 %)

Reset answer

```
ft=None, right=None):


S LINE! #########
```

שאלה **16**

לא נענה

ניקוד השאלה: 15.00

Time & Schedule

בשאלה זאת תממשו שתי מחלקות - **`Time`**, המתארת נקודת זמן ביממה, בדיוק של דקות, ו-**`Schedule`**, המתארת לוח זמנים של נסיעות רכבת.

במחלקה **`Time`** עליכם לממש את השיטות (Methods) הבאות:

• בנאי (Constructor) המחלקה אשר מקבל שני קלטים **`hours`** ו-**`minutes`**, שניהם מסוג int. ומייצר "זמן" חדש על ידי השעה והדקה הנתונות. על השעה להיות בתחום 0-23 (כולל) ועל הדקה להיות בתום 0-59 (כולל).
ניתן להניח שהטיפוסים תקינים אבל לא שהם בטווח החוקי. במידה ומתקבל קלט שאינו בטווח חוקי יש לזרוק שגיאה מסוג **`ValueError`**.

• מתודה שבהינתן ונרצה להדפיס את אוביקטים של המחלקה, יודפס הפורמט הבא:

`"<hours>:<minutes>"`

כך שבמקום **`<hours>`** ו-**`<minutes>`** יהיו הערכים המתאימים לאוביקט הזמן הספציפי. לדוגמה **13:57** או **2:46**.

• המחלקה מקושטת בדקורטור **`@total_ordering`** מתוך **`functools`**. דקורטור זה מאפשר להגדיר סדר בין אוביקטים השייכים למחלקה, ללא צורך בכתיבת פונקציות להגדרת ששת כיווני ההשוואה האפשריים (שווה/שונה/גדול/גודל או שווה/קטן/קטן או שווה) בין כל שני אוביקטים של המחלקה.
הדקורטור כבר מוגדר ועליכם לממש פונקצית שויון ופונקצית השוואה נוספת, לבחירתכם, כך שיהיה ניתן להשוות בין אובייקטים מסוג **`Time`**.

במחלקה **`Schedule`** עליכם לממש את השיטות (Methods) הבאות:

• בנאי (Constructor) המחלקה אשר לא מקבל קלטים, ומאתחל אוביקט לוח רכבות חדש, אשר בו ניתן להכניס זמני נסיעה. הלוח מאותחל להיות ללא זמני יציאה בכלל.

• המתודה **`add_time(self, t)`** אשר מקבלת קלט יחיד **`t`** מסוג **`Time`**, ומוסיפה את הזמן **`t`** להיות זמן יציאה נוסף ללוח הרכבות.
במידה ויש כבר רכבת שיוצאת באותה השעה - יש לזרוק שגיאה כללית מסוג **`Exception`** עם הודעת השגיאה:

`"The schedule already has a train leaving at time <hours>:<minutes>"`

כך שבמקום **`<hours>`** ו-**`<minutes>`** יהיו הערכים המתאימים לאוביקט הזמן הספציפי.

• המתודה **`find_train(self, t)`** אשר מקבלת קלט יחיד **`t`** מסוג **`Time`**, ומחזירה את זמן היציאה, אוביקט מסוג **`Time`**, המתאים לאדם שהגיע לתחנה בזמן **`t`**, כלומר זמן היציאה המוקדם ביותר שמגיע אחרי **`t`**.
הערות בעבור שיטה זו:
- אם ישנה רכבת שיוצאת בדיוק בזמן **`t`**, לא ניתן לתפוס אותה.
- אם אין אף רכבת היוצאת באותו היום אחרי זמן **`t`**, במידה ויש לתפוס זמן אחד אחר בלו"ז יש לתפוס את הרכבת הראשונה של היום הבא בשעה הכי

- אם אין אף-דרך בגודל n שמקיימת את התנאי, נחזיר None.
מוקדמת, אחרת - יש לזרוק שגיאה עם הודעה מתאימה.
- בשאלה זאת נדרש שזמן הריצה של המתודה יהיה $O(n \log(n))$ עבור n רכבות בלוח הזמנים (אין חשיבות לזמן הריצה של יתר השיטות).

Answer:

(penalty regime: 0 %)

Reset answer

```
ering


inutes):



be compared

as string
```

שאלה **17**

לא נענה

ניקוד השאלה: 1.00

סמנו איזה שאלות בחרתם בחלק ב', השאלות שתסמנו הן אלו שיבדקו **בלבד**.

- שאלה 11 - סכום רשימות
- שאלה 12 - סוגריים מאוזנים
- שאלה 13 - גודל הכתם
- שאלה 14 - חלוקת מספרים לזוגות
- שאלה 15 - עץ חיפוש בינארי
- שאלה 16 - Time & Schedule

שאלה **18**

לא נענה

לא ניתן ציון

תוכלו לרשום כאן הערות וטקסט חופשי במידת הצורך. **חלק זה אינו נבדק מראש** אלא רק במקרים חריגים ובמקרה של ערעורים.

